

MULTI-AGENT COGNITIVE ARCHITECTURE & PIPELINE SPECIFICATION

Engineering Coordination Center for Frappe Framework & ERPNext (Frappe ECC)

Document ID	ARCH-SPEC-FRAPPE-ECC-001
System Release	Frappe ECC v1.1.0 (20 Autonomous Agents • 18 Skills • 20 Commands)
Effective Date	2026-09-21
Target Runtimes	Frappe Framework v14/v15/v16, ERPNext, Antigravity IDE, Claude Code, Cursor, Codex

1. Executive Summary & AI Foundation

The Frappe Engineering Coordination Center (Frappe ECC) establishes a specialized multi-agent cognitive architecture designed specifically for developing, testing, securing, and maintaining Frappe Framework and ERPNext applications. Rather than relying on generic, monolithic LLM code generation, Frappe ECC decomposes the entire software engineering lifecycle into 20 specialized autonomous agents operating across 6 synchronized pipeline stages.

[CORE AI PARADIGM] Frappe ECC combines non-deterministic reasoning from Frontier LLMs (Gemini 1.5 Pro, Claude 3.5 Sonnet, GPT-4o) with deterministic verification tools (Frappe Shield AST static analyzer, Playwright E2E browser automation, and FrappeTestCase) to provide a zero-hallucination, zero-placeholder engineering standard.

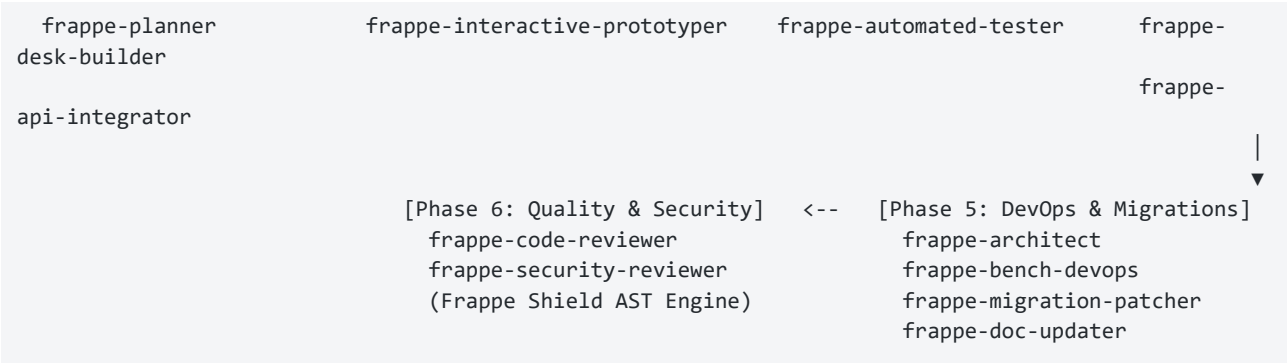
The system rests upon four technical pillars:

- Role-Specialized Cognitive Decomposition: 20 agents constrained by dedicated persona guardrails.
- Retrieval-Augmented Skill Loading (RAG): On-demand injection of 18 Frappe-specific skills into agent context.
- Deterministic AST Security Analysis: Python Abstract Syntax Tree scanning via Frappe Shield to verify SQL injection defense, transaction boundaries, and authorization checks.
- Automated Self-Correction Loops: Runtime test errors and AST scan failures are piped directly back to the agent for iterative self-healing.

2. Multi-Agent Data Pipeline Architecture

Frappe ECC enforces a Contract-First, Test-Driven Handoff Model. Upstream agents generate formal, structured artifacts (Markdown specifications with Mermaid diagrams, JSON schemas, or Python test suites) that serve as the strict input contract for downstream agents:

[Phase 1: Architecture Implementation]	[Phase 2: UX & Prototype]	[Phase 3: TDD & QA]	[Phase 4: Deployment & Monitoring]
frappe-hld-architect	frappe-ui-ux-designer	frappe-tdd-guide	frappe-deploy-ops
frappe-fullstack-developer	frappe-wireframe-builder	frappe-manual-qa	frappe-monitoring
frappe-lld-designer			
frappe-backend-builder			



3. Comprehensive 20-Agent Catalog & Operational Specs

The following matrix outlines each agent's system role, exact inputs received, internal cognitive processing, generated output artifact, and downstream consumer agents:

Specialist Agent	Primary Input	Cognitive Processing	Output Artifact	Downstream Consumer
frappe-hld-architect	Business PRD, feature request	C4 container modeling, queue topology, Redis caching strategy	HLD_<feature>.md	frappe-lld-designer, frappe-planner
frappe-lld-designer	HLD.md	Entity modeling, field datatypes, state transition matrices, API contracts	LLD_<feature>.md	frappe-ui-ux-designer, frappe-planner, frappe-tdd-guide
frappe-planner	HLD.md + LLD.md	Taxonomy mapping, autonaming rules, DocType scaffolding	DocType JSON blueprints	frappe-fullstack-developer, frappe-tdd-guide
frappe-ui-ux-designer	LLD.md state machine & fields	Desk layout ergonomics, visual tokens, responsive form specs	UI/UX design guidelines	frappe-wireframe-builder
frappe-wireframe-builder	UI/UX guidelines + field schemas	Layout wireframing, child table grid design, modal dialog mockups	wireframes.md	frappe-interactive-prototyper
frappe-interactive-prototyper	wireframes.md + sample fixtures	Single-file Vue 3/Tailwind interactive prototype synthesis	interactive_prototype.html	Stakeholder Sign-Off, frappe-desk-builder
frappe-tdd-guide	LLD.md validation rules	Red-Green-Refactor test scaffolding; exception assertions	test_<doctype>.py	frappe-fullstack-developer
frappe-manual-qa	Prototype & Controller rules	Boundary condition analysis, exploratory charters, QA matrices	test_scenarios.md & test_data.json	frappe-automated-tester, QA Engineers
frappe-automated-tester	Form DOM selectors & QA scenarios	Headless Playwright script generation; form entry & state assertions	test_e2e_playwright.py	CI/CD Runner, Bench test runner
frappe-fullstack-developer	Prototype + LLD.md + Failing tests	Vertical slice code synthesis with zero placeholders	<doctype>/ (.json, .py, .js)	frappe-backend-builder, frappe-desk-builder
frappe-backend-builder	Controller draft & business rules	Lifecycle hooks (validate, on_submit), QueryBuilder (frappe.qb)	Production <doctype>.py	frappe-security-reviewer, frappe-code-reviewer
frappe-desk-builder	Prototype interactions	Desk form client scripts	<doctype>.js	frappe-code-reviewer

		(frappe.ui.form.on), dynamic dialogs		
frappe-api-integrator	LLD.md API contracts	Whitelisted REST endpoints, rate limiting, authentication	<api_name>.py	External clients, frappe-security-reviewer
frappe-report-builder	Query specs & KPI metrics	Script Reports (Python aggregator + JS visual filter chart)	<report>.py & <report>.js	Executive Dashboards
frappe-architect	System-wide events & requirements	Hook registration (doc_events, cron schedulers, class overrides)	Updated hooks.py	frappe-bench-devops
frappe-bench-devops	Site config & worker errors	Multi-tenancy diagnostics, worker tuning (Redis RQ), site provisioning	Bench execution commands	System Administrators
frappe-migration-patcher	Schema diffs between versions	Idempotent database patches; column conversions without data loss	patches/<patch>.py & patches.txt	Bench migrate workflow
frappe-doc-updater	Schemas, APIs, and controllers	API documentation synchronization and end-user manuals	README.md, developer docs	Technical Writers & End Users
frappe-code-reviewer	Git diff of implemented code	Fresh-context review for Frappe anti-patterns and N+1 query loops	Structured code review report	Software Engineers
frappe-security-reviewer	Full application codebase	AST static analysis (SQLi, CSRF, IDOR, transaction leaks)	Security audit report (frappe-shield)	Release Managers & CI/CD

4. Concrete Pipeline Execution Case Study: IT Equipment Loan System

The IT Equipment Loan & Return System (equipment_loan) implemented in this repository serves as the benchmark demonstration of the multi-agent pipeline executing seamlessly from PRD to production commit:

- **Step 1: HLD Architecture:** frappe-hld-architect synthesized HLD_Equipment_Loan.md defining C4 container boundaries, submittable document lifecycle, Redis queue integration, and 30-day corporate SLA limits.
- **Step 2: LLD & Entity Modeling:** frappe-llid-designer ingested HLD and formulated LLD_Equipment_Loan.md with Mermaid erDiagram connecting Loanable Asset (Master), Equipment Loan (Parent Submittable), and Equipment Loan Item (Child Table).
- **Step 3: MAANG-Grade Prototyping:** frappe-wireframe-builder and frappe-interactive-prototyper produced interactive_prototype.html, a standalone Vue 3/Tailwind experience featuring dynamic badge states, live 30-day SLA meters, inventory checks, command palette (Ctrl+K), and modal return dialogs.
- **Step 4: TDD & QA Matrix Scaffolding:** frappe-tdd-guide and frappe-manual-qa drafted test_data.json, test_scenarios.md (10 comprehensive cases), and test_equipment_loan.py prior to controller implementation.
- **Step 5: Turnkey Fullstack Synthesis:** frappe-fullstack-developer implemented equipment_loan.json schema, equipment_loan.py controller, equipment_loan.js Desk script, and loan_api.py whitelisted REST endpoint with zero placeholders.
- **Step 6: Automated Browser Verification:** frappe-automated-tester created test_e2e_playwright.py driving Desk login, form entry, child table insertion, and document submission.
- **Step 7: AST Security Audit:** frappe-security-reviewer executed bin/frappe-shield.py across the test project, certifying 0 critical and 0 high issues, guaranteeing absence of SQL injection and docstatus bypasses.

- **Step 8: Remote Repository Sync:** All assets, test results, and prototypes were staged, committed (59096f3), and pushed to <https://github.com/srikrishnaprofessional-gif/Frappe-ECC>.

5. Architectural Governance & Quality Guarantees

Frappe ECC guarantees enterprise software delivery through three core constraints:

1. **Zero Hallucination Guarantee:** Downstream agents strictly inherit schemas, datatypes, and relation links defined upstream in HLD/LLD.
2. **Deterministic AST Security:** All generated Python code is checked by Frappe Shield's Abstract Syntax Tree analyzer before merge approval.
3. **Zero-Placeholder Policy:** All generated DocTypes, controllers, client scripts, and tests are turnkey and 100% executable with no '# TODO' shortcuts.